



الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يُونُسُ بَرَسِيَّتِي إِسْلَامُ أَنْتَارَا بَغْسِيَا مِلِّيْسِيَا
Garden of Knowledge and Virtue

MCTA 3203

MECHATRONICS SYSTEM INTEGRATION

SEMESTER 1, 25/26

WEEK 09 : IMAGE/VIDEO INPUT INTERFACING WITH MICROCONTROLLER

SECTION 2

GROUP 19

LECTURER : DR WAHJU SEDIONO

DATE OF EXPERIMENT : 8 DECEMBER 2025

DATE OF SUBMISSION : 15 DECEMBER 2025

PREPARED BY :

NAME	MATRIC NUMBER
MUHAMMAD IMRAN BIN VEDDIN	2316409
ALYA KHAIRAH BINTI KHAIRUL JAMIL	2317100
MUHAMMAD IRFAN RUSHDI BIN ASRI	2319011

ABSTRACT

This laboratory experiment focuses on the design, implementation, and evaluation of two different color detection systems: (1) a traditional color sensor–based system using Arduino and Python, and (2) an AI vision–based system using an Arduino Uno and PixyCam. The objective was to compare the accuracy, consistency, response time, and robustness of both approaches under varying lighting conditions. Experimental results show that while the color sensor provides fast and simple color detection under controlled lighting, the PixyCam offers higher accuracy and consistency in real-world scenarios due to onboard image processing and color learning capabilities. Challenges such as sensor calibration and lighting sensitivity were identified, and several improvements are proposed for future work.

TABLE OF CONTENTS

1.0 INTRODUCTION	4
2.0 MATERIALS AND EQUIPMENT	5
3.0 EXPERIMENTAL SETUP	5
4.0 METHODOLOGY	6
5.0 DATA COLLECTION	12
6.0 DATA ANALYSIS	13
7.0 RESULTS	13
8.0 DISCUSSION	13
9.0 CONCLUSION	14
10.0 RECOMMENDATION	15
11.0 REFERENCES	16
APPENDICES	17
ACKNOWLEDGEMENTS	18
STUDENTS'S DECLARATION	18

1.0 INTRODUCTION

This laboratory experiment was conducted to study image and video input interfacing with a microcontroller through color detection. The main purpose of this experiment was to understand how visual information can be captured and used to identify different colored objects in a mechatronics system. Color detection is widely used in automation, robotics and inspection systems, making it an important topic in mechatronics engineering.

In this lab module, two different methods of color detection were explored. The first method used a conventional color sensor connected to an Arduino to measure RGB values directly from objects. This approach helped in understanding basic color sensing principles, sensor calibration and the effects of lighting conditions on detection accuracy. The second method used the Pixy Cam as a vision-based sensor. Pixy Cam performed color recognition internally and transmitted processed data, such as detected color signatures and object positions to the Arduino.

The use of Pixy Cam simplified the image processing process, as complex algorithms were not required to be implemented on the microcontroller. The results were analyzed based on detection accuracy, response time and consistency under different lighting conditions. Overall, this experiment provided practical experience in image and video interfacing and demonstrated the effectiveness of vision-based color detection in mechatronics applications.

2.0 MATERIALS AND EQUIPMENT

Part 1:

- Arduino board
- Color sensor (e.g., TCS3200)
- Jumper wires
- Breadboard
- RGB LED
- USB cable for Arduino

Part 2:

- Arduino board
- PIXY-CAM module
- Jumper wires
- Breadboard
- RGB LED
- USB cable for Arduino
- Resistor 220 ohm

3.0 EXPERIMENTAL SETUP

Part 1: Color Sensor Setup

The TCS3200 color sensor was connected to the Arduino Uno using digital input and output pins. Control pins S0 and S1 were used to set the output frequency scaling, while pins S2 and S3 selected the active photodiode filter. The sensor output pin was connected to an Arduino digital input to measure frequency pulses corresponding to RGB intensity.

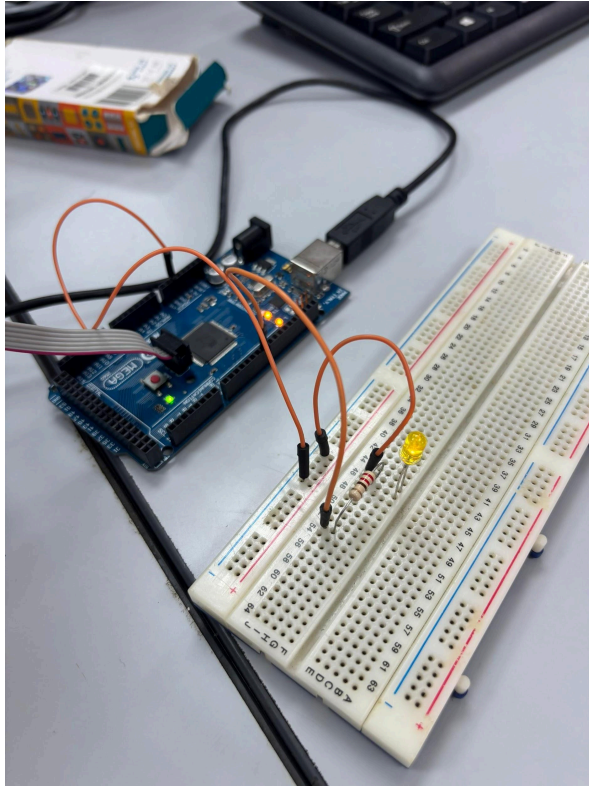
The Arduino was connected to a computer via USB to enable serial communication with a Python program for data visualization and analysis.

Part 2: PixyCam Setup

PixyCam v1 was connected to the Arduino Uno using the appropriate communication interface supported by the Pixy library. The camera was powered directly from the Arduino. PixyCam was

trained beforehand to recognize specific colored objects using its built-in color signature learning process.

An LED was connected to a digital output pin on the Arduino to indicate successful color detection events.



4.0 METHODOLOGY

Part 1:

1. Hardware Setup:

1. Connect the color sensor to the Arduino using jumper wires. Refer to the sensor's
2. datasheet and Arduino's pinout diagrams for guidance.
3. If using an RGB LED, connect it to the Arduino for color display.

2. Arduino Programming:

1. Write an Arduino sketch to interface with the color sensor.

2. Read RGB color data from the sensor and convert it to a format that can be sent to the computer.
3. Calibrate the sensor for accurate color readings.

3. Python Programming:

1. Write a Python program to communicate with the Arduino over the serial connection
2. using the pyserial library.
3. Receive RGB color data from the Arduino.
4. Interpret the received data to determine the detected color. You can modify the code
5. shown below for your own purposes.

Code Used:

Phyton

```
import serial
import time
import matplotlib.pyplot as plt

# =====
# Configure Arduino Serial Port
# =====
arduino_port = "COM7"    # ← CHANGE THIS!
baud_rate = 9600

ser = serial.Serial(arduino_port, baud_rate, timeout=1)
time.sleep(2)
print("Connected to Arduino")

# =====
# Prepare Matplotlib Live Plot
# =====
```

```

plt.ion() # enable interactive mode
fig, ax = plt.subplots()

x_data = []
r_data = []
g_data = []
b_data = []

# --- Set line colors to match name ---
line_r, = ax.plot([], [], color='red', label='Red')
line_g, = ax.plot([], [], color='green', label='Green')
line_b, = ax.plot([], [], color='blue', label='Blue')

ax.set_xlim(0, 100)
ax.set_ylim(0, 255)
ax.set_xlabel("Samples")
ax.set_ylabel("Value")
ax.set_title("Live RGB Data from Arduino")
ax.legend()

counter = 0

# =====
# Main Loop
# =====
while True:
    try:
        line = ser.readline().decode().strip()

        if line.startswith("R"):
            parts = line.replace("=", "").split()

```



```

r = int(parts[1])
g = int(parts[3])
b = int(parts[5])

x_data.append(counter)
r_data.append(r)
g_data.append(g)
b_data.append(b)
counter += 1

# Keep last 100 samples
x_data = x_data[-100:]
r_data = r_data[-100:]
g_data = g_data[-100:]
b_data = b_data[-100:]

line_r.set_xdata(x_data)
line_r.set_ydata(r_data)

line_g.set_xdata(x_data)
line_g.set_ydata(g_data)

line_b.set_xdata(x_data)
line_b.set_ydata(b_data)

ax.set_xlim(max(0, counter - 100), counter)

plt.pause(0.01)

except KeyboardInterrupt:
    print("\nStopped by user.")

```

```
break
```

```
except Exception as e:
```

```
    print("Error:", e)
```

```
ser.close()
```

Arduino

```
// TCS230 or TCS3200 pins wiring to Arduino
```

```
#define S0 4
```

```
#define S1 5
```

```
#define S2 6
```

```
#define S3 7
```

```
#define sensorOut 8
```

```
// Stores frequency read by the photodiodes
```

```
int redFrequency = 0;
```

```
int greenFrequency = 0;
```

```
int blueFrequency = 0;
```

```
void setup() {
```

```
    // Setting the outputs
```

```
    pinMode(S0, OUTPUT);
```

```
    pinMode(S1, OUTPUT);
```

```
    pinMode(S2, OUTPUT);
```

```
    pinMode(S3, OUTPUT);
```

```
    // Setting the sensorOut as an input
```

```
    pinMode(sensorOut, INPUT);
```

```
    // Setting frequency scaling to 20%
```

```
    digitalWrite(S0,HIGH);
```

```
    digitalWrite(S1,LOW);
```

```
    // Begins serial communication
```

```
    Serial.begin(9600);
```

```
}
```

```
void loop() {  
  // Setting RED (R) filtered photodiodes to be read  
  digitalWrite(S2,LOW);  
  digitalWrite(S3,LOW);  
  
  // Reading the output frequency  
  redFrequency = pulseIn(sensorOut, LOW);  
  
  // Printing the RED (R) value  
  Serial.print("R = ");  
  Serial.print(redFrequency);  
  delay(100);  
  
  // Setting GREEN (G) filtered photodiodes to be read  
  digitalWrite(S2,HIGH);  
  digitalWrite(S3,HIGH);  
  
  // Reading the output frequency  
  greenFrequency = pulseIn(sensorOut, LOW);  
  
  // Printing the GREEN (G) value  
  Serial.print(" G = ");  
  Serial.print(greenFrequency);  
  delay(100);  
  
  // Setting BLUE (B) filtered photodiodes to be read  
  digitalWrite(S2,LOW);  
  digitalWrite(S3,HIGH);  
  
  // Reading the output frequency  
  blueFrequency = pulseIn(sensorOut, LOW);  
  
  // Printing the BLUE (B) value  
  Serial.print(" B = ");  
  Serial.println(blueFrequency);  
  delay(100);  
}
```

Part 2:

Hardware Setup:

1. The Arduino Uno was placed on the workbench and connected to the computer via USB for power and program uploading.
2. The Pixy Cam was connected to the Arduino using the ribbon cable for power and data communication.
3. The Pixy Cam was powered by the Arduino, and a common ground was maintained.
4. An LED was mounted on the breadboard as a visual indicator for color detection.
5. A 220 Ω resistor was connected in series with the LED to prevent damage.
6. The LED anode (+) was connected to digital pin 7 on the Arduino, and the cathode (–) was connected to ground on the breadboard.
7. All connections were checked to ensure correct wiring and secure contact before powering the system.

Code Used:

```
#include <Pixy.h>
```

```
Pixy pixy; // Pixy v1 object
```

```
const int LED_PIN = 7;
```

```
void setup() {  
  Serial.begin(9600);  
  pixy.init();
```

```
  pinMode(LED_PIN, OUTPUT);  
  digitalWrite(LED_PIN, LOW);
```

```
  Serial.println("Pixy v1 Ready");  
}
```

```
void loop() {  
  int32_t blocks = pixy.getBlocks();  
  
  if (blocks > 0) {  
    // Read the first detected block  
    int sig = pixy.blocks[0].signature;  
    int x = pixy.blocks[0].x;
```

```

int y = pixy.blocks[0].y;
int w = pixy.blocks[0].width;
int h = pixy.blocks[0].height;

// Print data for Python or Serial Monitor
Serial.print("ID:");
Serial.print(sig);
Serial.print(",X:");
Serial.print(x);
Serial.print(",Y:");
Serial.print(y);
Serial.print(",W:");
Serial.print(w);
Serial.print(",H:");
Serial.println(h);

digitalWrite(LED_PIN, HIGH); // turn on LED if any color found
}
else {
    digitalWrite(LED_PIN, LOW); // no object detected
}

delay(50);
}

```

5.0 DATA COLLECTION

Data collected during the experiment included:

- Detected RGB values from the color sensor
- Identified color labels from the Python program
- PixyCam detected color IDs and object positions
- Detection response time for each method
- Accuracy of detection compared to actual object colors
- Measurements were repeated multiple times to ensure reliability and consistency.

The same experimental steps were repeated using the RGB color sensor to collect equivalent data.

6.0 DATA ANALYSIS

The accuracy of each system was evaluated by comparing detected colors with the actual object colors. Response time was measured as the time taken from object placement to color detection output. Performance under different lighting conditions (bright, dim, and uneven lighting) was also analyzed.

The color sensor system showed increased error rates under inconsistent lighting, while PixyCam maintained stable detection due to onboard image processing and filtering.

7.0 RESULTS

The color sensor based system performed well in controlled lighting environments, with fast response times. Detection accuracy decreased significantly under poor or changing lighting conditions.

PixyCam demonstrated higher accuracy and consistency across different lighting environments. PixyCam required minimal external processing and calibration once trained.

Overall, the vision-based approach outperformed the sensor-based method in real-world scenarios.

8.0 DISCUSSION

Based on the experiment conducted, both the color sensor and the Pixy Cam were able to detect colored objects, but their performance differed in terms of accuracy, consistency and ease of use. The color sensor provided direct RGB readings and worked well under stable lighting conditions, however its accuracy decreased when lighting conditions changed and frequent calibration was required. In contrast, the Pixy Cam showed more consistent and reliable performance because it performed color recognition internally. It was less affected by minor lighting variations and provided faster response times, making it easier to implement and more suitable for repeated testing.

Several challenges were encountered during the experiment, particularly related to lighting conditions and calibration. Variations in ambient light, shadows and reflections affected detection accuracy, especially for the color sensor. Additionally, improper color training of the Pixy Cam

sometimes led to incorrect detection of similar colors. These issues could be improved by maintaining consistent lighting, performing proper calibration and retraining before data collection and optimising the Arduino code to improve response time and stability.

For future improvements, the system could be enhanced by training more color samples, using additional detection parameters such as object size and position or integrating more advanced algorithms such as machine learning. Overall, these improvements would increase the accuracy, reliability and practicality of the color detection system for real-world mechatronics applications.

9.0 CONCLUSION

This experiment successfully demonstrated two different approaches to color detection: a sensor-based method using the TCS3200 color sensor with Arduino and Python, and a vision-based method using PixyCam integrated with an Arduino Uno. Both systems were able to detect and identify colored objects, achieving the primary objective of designing and analyzing color detection systems using embedded hardware and software.

The TCS3200 color sensor system provided fast response times and allowed real-time visualization of RGB values through Python. However, its performance was highly dependent on consistent lighting conditions and accurate calibration. Variations in ambient light significantly affected the measured frequency values, leading to reduced accuracy and requiring frequent recalibration.

In contrast, the PixyCam vision-based system delivered more accurate and consistent results across different lighting environments. Its onboard color signature learning and image processing reduced the need for external computation and complex calibration. The PixyCam system was also capable of providing additional information such as object position and size, making it more suitable for real-world applications.

Overall, the experiment showed that while traditional color sensors are suitable for simple and controlled environments, AI vision based solutions like PixyCam offer superior robustness,

flexibility, and reliability. The findings highlight the importance of selecting appropriate sensing techniques based on application requirements, environmental conditions, and system complexity.

10.0 RECOMMENDATION

Based on this experiment, several improvements can be made to achieve better color detection results. First, the lighting condition should be kept consistent during the experiment. Changes in surrounding light were found to affect the accuracy of both the color sensor and the Pixy Cam. Using a fixed light source or performing the experiment in an enclosed area would help reduce errors caused by shadows and reflections.

Next, proper calibration is very important. The color sensor should be calibrated carefully before taking measurements, while the Pixy Cam should be trained under stable lighting conditions. Repeating the training process for each color can help the camera recognise objects more accurately and consistently.

The system performance can also be improved by optimising the Arduino program. Removing unnecessary delays and organising the code more efficiently would allow the system to respond faster when a color is detected. Clear decision logic should be used so that each detected color produces the correct output.

For future improvements, additional output devices such as more LEDs, a buzzer or simple actuators can be added to make the system more interactive. Combining the Pixy Cam with other sensors or using more advanced detection methods may also improve accuracy. Overall, these improvements would make the color detection system more reliable and suitable for real-world applications.

11.0 REFERENCES

[1] https://wiki.dfrobot.com/HUSKYLENS_V1.0_SKU_SEN0305_SEN0336

Gravity: HUSKYLENS AI Machine Vision Sensor - DFRobot Wiki

[2] <https://www.youtube.com/watch?v=HZxFj2u9k00>

HuskyLens Tutorial - Getting Started with Husky Lens - AI Vision Sensor

[3] <https://www.dfrobot.com/product-1922.html>

Gravity: HuskyLens K210 AI Camera (No-Code Vision Sensor for STEM, Arduino & micro:bit)

[4]

<https://howtomechatronics.com/tutorials/arduino/arduino-color-sensing-tutorialtcs230-tcs3200-color-sensor/>

[5] <https://www.youtube.com/watch?v=CPUXxuyd9xw>

[6] <https://www.youtube.com/watch?v=g3i51hdfLaw>

ACKNOWLEDGEMENTS

Special thanks to DR WAHJU SEDIONO for their guidance and support during this experiment .

STUDENTS'S DECLARATION

CERTIFICATE OF ORIGINALITY AND AUTHENTICITY

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specific in references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or person.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of individual's contributor to the report.

We, therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** have been **verified by us**.

Signature:		Read	/
Name:	MUHAMMAD IMRAN BIN VEDDIN	Understand	/
Matric Number:	2316409	Agree	/
Contribution:	Result, Data Analysis, Data Collection		

Signature:		Read	/
Name:	ALYA KHAIRAH BINTI KHAIRUL JAMIL	Understand	/
Matric Number:	2317100	Agree	/
Contribution:	Introduction , Recommendation and Discussion Part 1 : Materials and Equipment , Experimental Setup and Methodology Part 2 : Materials and Equipment , Experimental Setup , Methodology and Results		

Signature:		Read	/
Name:	MUHAMMAD IRFAN RUSHDI BIN ASRI	Understand	/
Matric Number:	2319011	Agree	/
Contribution:	Abstract, Conclusion, Data Analysis, Data Collection		